



2



12



1440 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

مفاهيم البرمجة بلغة Java

د. محمد الأحمد / د. عبد الرحمن اللحام



RB Informatics; 16/04/2025

البرمجة 2

أساسيات برنامج Java

✚ هيكل البرنامج الأساسي:

- كل برنامج Java يتكون من class واحد على الأقل تقوم أنت بتعريفه

✚ Class: هو المخطط أو النموذج الذي يتم من خلاله إنشاء الكائنات (objects). يجب

أن يعلن عن ال public class في ملف يحمل نفس اسم ال class بامتداد .java.

- على سبيل المثال: إذا كان اسم الكلاس Welcome1، فيجب حفظ الملف باسم Welcome1.java

✚ Main method: هي نقطة البداية لتنفيذ أي تطبيق java. يتم تعريفها بالشكل

التالي:

```
public static void main(String [] args).
```

✚ حساسية حالة الأحرف (case sensitivity):

لغة جافا حساسة لحالة الأحرف، مما يعني أن Welcome تختلف عن .welcome.

✚ مثال 1:

✚ برنامج طباعة بسيط:

```
public class Welcome1
{
    public static void main(String[] args)
    { System.out.println("Welcome to Java Programming!");
    } // end method main
} // end class Welcome
```

■ السطر 1: عرفنا class اسمه Welcome1.

■ السطر 2: الدالة الرئيسية.

■ السطر 3: هي جملة تقوم بطباعة النص الموجود بين علامتي التنصيص ثم تنتقل إلى سطر جديد.

أوامر الطباعة:

`System.out.println()`: تطبع السلسلة النصية (String) ثم تنقل المؤشر إلى بداية السطر الثاني.
`System.out.print()`: تطبع السلسلة النصية ويبقى المؤشر في نفس السطر بعد النص المطبوع.
`System.out.printf()`: تسمح بطباعة نص منسق، حيث يمكن استخدام محددات تنسيق (format specifiers) `%d` للسلاسل النصية و `%s` للأعداد الصحيحة.

مثال 2: استخدام `System.out.print()` و `System.out.println()`

```
public class Welcome2
{
    public static void main(String[] args)
    {
        System.out.print("Welcome to "); // تطبع ويبقى المؤشر في نفس السطر
        System.out.println("Java Programming!"); // تطبع وتنقل لسطر جديد
    }
}
```

الخرج:

Welcome to java programming

مثال 3: استخدام `System.out.printf()`

```
public class Welcome4 {
    public static void main(String[] args)
    {
        System.out.printf("%s\n%s\n", "Welcome to", "Java Programming!");
    }
}
```

الخرج:

Welcome to
Java programming



Escape Sequences:

- \n (new line) سطر جديد. ينقل المؤشر إلى بداية السطر التالي.
- \t (Horizontal tab) مسافة جدول أفقية. ينقل المؤشر إلى نقطة الجدولة التالية.
- \r (Carriage return) عودة إلى بداية السطر. ينقل المؤشر إلى بداية السطر الحالي.
- \\ لطباعة الشرطة المائلة العكسية \
- \" لطباعة علامة التنصيص المزدوجة "

❖ مثال 4: طباعة عدة أسطر باستخدام \n :

```
public class Welcome3
{
    public static void main(String[] args)
    {
        System.out.println("Welcome\nto\nJava\nProgramming!");
    }
}
```

❖ الخرج:

Welcome
To
Java
Programing!

إدخال البيانات والمتغيرات:

❖ Import declaration (إعلانات الاستيراد):

تساعد المترجم (compiler) في تحديد موقع class معرف مسبقاً يستخدم في برنامج java. توفر مجموعة غنية من الكلاسات المعرفة مسبقاً (Java API) التي يمكن إعادة استخدامها. تجمع الكلاسات في مجموعات مسماة من الكلاسات المرتبطة (packages).

وتحتوي على كلاسات أساسية، هذه الحزمة يتم استيرادها ضمناً في كل برنامج Java: java.lang .
مثل System.

استخدام scanner لإدخال البيانات:

كلاس Scanner يمكن البرنامج من قراءة البيانات من مصادر مختلفة، مثل إدخال المستخدم من لوحة المفاتيح. قبل استخدام Scanner، يجب إنشاؤه وتحديد مصدر البيانات.

```
import java.util.Scanner; // استيراد كلاس Scanner
```

```
Scanner input = new Scanner(System.in); // للقراءة من لوحة المفاتيح إنشاء كائن
```

الكلمة المفتاحية new تستخدم لإنشاء كائن.

- علامة المساواة (=) في الإعلان تشير إلى أن المتغير يجب أن يتم تهيئته أي إعداده للاستخدام في البرنامج (مع نتيجة التعبير الموجود على يمين علامة المساواة).
- System.in هو كائن يستخدم لقراءة البيانات المدخلة من قبل المستخدم. يمكن أن تكون هذه البيانات نصوصاً أو أرقاماً أو أي نوع آخر من المعلومات.
- كائن Scanner هو أداة تستخدم لتحويل البيانات المدخلة (التي تمثل بايتات) إلى أنواع بيانات يمكن التعامل معها بسهولة في البرنامج، مثل الأعداد الصحيحة أو السلاسل النصية.

ملاحظات خارجية:

- nextInt(): تقرأ قيمة صحيحة من المستخدم (int).
- بالنسبة لباقي الأنواع من المتغيرات:
- nextDouble(): قراءة الأعداد العشرية (double).
- nextFloat(): قراءة الأعداد العشرية (float).
- nextBoolean(): قراءة القيم المنطقية (boolean).
- Next(): قراءة سلسلة نصية (string)، لقراءة كلمة واحدة حتى (المسافة).
- nextLine(): لقراءة سطر كامل.

🔗 مثال 5: برنامج جمع عددين:

```
import java.util.Scanner; // program uses class Scanner
```

```
{
    public static void main(String[] args)
    {
        Scanner input = new Scanner(System.in); // create a Scanner

        int number1; // first number to add
        int number2; // second number to add
        int sum; // sum of number1 and number2

        System.out.print("Enter first integer: "); // prompt
        number1 = input.nextInt(); // read first number from user
        System.out.print("Enter second integer: "); // prompt
        number2 = input.nextInt(); // read second number from user
        sum = number1 + number2; // add numbers
    }
}
```

```
System.out.printf("Sum is %d\n", sum); // display sum
} // end method main
} // end class Addition
```

➤ الخرج:

Enter first integer: 45
Enter second integer: 72
Sum is 117

العمليات الحسابية وعمليات المقارنة:

➤ العمليات الحسابية:

- +: الجمع
- -: الطرح
- *: الضرب
- /: القسمة
- %: باقي القسمة

➤ أسبقية العمليات الحسابية:

- يتم تقييمها من اليسار إلى اليمين إذا تساوت الأسبقية. (*, /, %).
- يتم تقييمها من اليسار إلى اليمين إذا تساوت الأسبقية. (+, -).
- الإسناد يتم تقييمها أخيراً (=)

➤ عمليات المساواة والعلاقات:

- ==: يساوي (Equality)
- !=: لا يساوي (not Equal)
- >: أكبر من
- <: أصغر من
- >=: أكبر من أو يساوي
- <=: أصغر من أو يساوي

■ تستخدم هذه العمليات عادة في جمل if الشرطية.



التحكم في سير البرنامج (Control Statements):

قواعد إنشاء الجمل:

- تنتهي كل جملة بسيطة بفاصلة منقوطة (;)

- يمكن تجميع عدة جمل داخل أقواس معقوفة ({}) لتشكيل كتلة (block).
- تستخدم الأقواس المعقوفة مع جمل التحكم.

↪ أنواع التحكم في السير الأساسية:

يمكن تصنيف التحكم في السير إلى أربعة أنواع:

1. Sequential: تسلسلي (تنفيذ جمل بترتيب كتابتها).
2. Selection: اختيار (تنفيذ جمل معينة بناءً على شرط).
3. Iteration: تكرار (تكرار تنفيذ جمل معينة عدة مرات).
4. Transfer: نقل (نقل التحكم إلى جزء آخر من البرنامج مثل (break, continue)).

↪ Creating Code Blocks:

Enclose all class declarations: عند تعريف كلاس جديد في جافا، يتم استخدام قوسين معقوفين {} لتحديد كتلة الفئة. كل ما يتعلق بالفئة، مثل المتغيرات والطرق، يجب أن يكون داخل هذه الكتلة.

Enclose all method declarations: كل طريقة تعرف أيضاً باستخدام قوسين معقوفين {}. الكتلة تحدد نطاق الطريقة، مما يعني أن المتغيرات المعرفة داخل هذه الكتلة لا يمكن الوصول إليها من خارجها.

Group other related code segments: يمكنك أيضاً استخدام الكتل لتنظيم أجزاء مختلفة من الكود التي تتعلق ببعضها البعض. على سبيل المثال، يمكنك إنشاء كتل لتجميع التعليمات المتعلقة بمعالجة البيانات أو تنفيذ العمليات المنطقية.

```
Public class SayHello {
    Public static void main (string [] args) {
        System.out.println("Hello world");
    }
}
```

نطاق المتغيرات (Scoping):

النطاق (scope) يحدد رؤية (visibility) ومدة حياة (lifetime) الأسماء، المتغيرات والدوال (المعرفة ضمن ذلك النطاق). عادة ما يتم تحديد النطاق بواسطة الأقواس المعقوفة {}.

- المتغير المعرفة داخل كتلة يكون متاحاً فقط داخل تلك الكتلة وبعدها يخرج من النطاق (out of scope)
- لا يمكن تعريف متغير بنفس الاسم في نطاق متداخل إذا كان الاسم معرّفاً بالفعل في النطاق الخارجي.
- بالنسبة للكائنات (objects)، عندما يخرج مرجع الكائن (object reference) مثل s من النطاق، فإن النطاق نفسه الذي يشير إليه s لا يزال يحتل مكان في الذاكرة حتى يقوم مجمع القمامة بإزالتها (garbage collector).

```
{
String s = new String("a string");
} /* end of scope */
```

Control Statements:

▪ If (single-selection Statement): تنفذ جملة أو كتلة من الجمل إذا كان الشرط صحيحاً. (true)

```
if (studentGrade >= 60)
    System.out.println("Passed");
```

▪ If...else (Double-Selection Statement): تنفذ جملة أو كتلة إذا كان الشرط صحيحاً، وتنفذ جملة أو كتلة أخرى إذا كان الشرط خاطئاً (false).

```
if (grade >= 60)
    System.out.println("Passed");
else
    System.out.println("Failed");
```

• يمكن عمل nested if...else جمل if...else متداخلة (لاختبار عدة حالات).

```
if (studentGrade >= 90)
    System.out.println("A");
else if (studentGrade >= 80)
    System.out.println("B");
// وهكذا ...
else
    System.out.println("F");
```

▪ **ملاحظة (note):** حول else المتدلية (Dangling else): في حالة if-else متداخلة، يتم ربط else دائماً بأقرب بعد، مالم تستخدم الأقواس {} لتغيير ذلك else لم يتم ربطها ب if

المعامل الشرطي (Conditional Operator/ Ternary Operator):

• هو اختصار لجملة if..else يأخذ ثلاث معاملات.

```
System.out.println(studentGrade >= 60 ? "Passed" : "Failed");
```

يكافئ:

```
If (studentGrade >= 60)
    System.out.println("Passed");
Else
    System.out.println("Failed");
```

Switch Statement:

- تستخدم لاختيار واحد من عدة مسارات تنفيذ بناءً على قيمة تعبير.
- تستخدم Case Labels لتحديد القيم المختلفة. يجب أن تكون هذه القيم ثوابت.
- تستخدم break للخروج من switch بعد تنفيذ case معينة. إذا لم تستخدم break، سيحدث "fall through" أي سيتم تنفيذ ال case التالية أيضاً.
- يفضل دائماً توفير default case ليتم تنفيذها إذا لم تتطابق أي من ال case labels مع قيمة التعبير

↪ مثال 6:

```
switch (gradeChar) {
    case 'A':
        System.out.println("Excellent");
        break;
    case 'B':
        System.out.println("Good");
        break;

    default:
        System.out.println("Invalid grade");
}
```

جمل التكرار (Repetition/ Looping Statements):

يوجد ثلاث أنواع وهي:

1. while
2. do ... while
3. for

↪ تستخدم لتكرار تنفيذ مجموعة من الجمل. لكل حلقة تكرار أربعة أجزاء أساسية:

1. Initialization: (تهيئة) إعداد متغير التحكم في الحلقة.
2. Iteration condition: (شرط التكرار) الشرط الذي يجب أن يكون صحيحاً لاستمرار الحلقة.
3. Body: (جسم الحلقة) الجمل التي يتم تكرارها.
4. Termination: (تحديث / إنهاء) تعديل متغير التحكم لضمان انتهاء الحلقة في النهاية.

-1 While loop: يتم اختبار الشرط قبل كل تكرار. إذا كان الشرط صحيحاً، يتم تنفيذ جسم الحلقة.

```

Int I = 0; //Initialization
While (I < 10) { // iteration condition
    System.out.println("i = " + i); // Body
    I++; // Termination (Update)
}

```

-2 do ... while Loop: يتم تنفيذ جسم الحلقة مرة واحدة على الأقل، ثم يتم اختبار الشرط. إذا كان الشرط صحيحاً، يستمر التكرار.

```

int i = 0; // Initialization
do {
    System.out.println("i = " + i); // Body
    i++; // Termination (Update)
} while (i < 10); // Iteration condition

```

-3 for loop: توفر طريقة مدمجة للتهيئة والشرط والتحديث في سطر واحد هي الأكثر شيوعاً بين حلقات التكرار.

⚡ **for (Initialization- Iteration condition- Iteration/Update) statement;**

```

for (int i = 0; i < 10; i++) {
    System.out.println("i = " + i);
}

```

كيف سيبدو الكود باستخدام حلقة while؟

```

int i = 0;
while (i < 10) {
    System.out.println("i = " + i);
    i++;
}

```

- يمكن تعريف المتغيرات في جزء التهيئة من حلقة for.
- يمكن أن تتكون التهيئة والتحديث من قائمة من التعبيرات مفصولة بفواصل.
- شكل الحلقة اللانهائية:

for(;;)

while(true)



Break and continue in loops:

يمكنك أيضاً التحكم في سير الحلقة داخل جسم أي من عبارات التكرار باستخدام break و continue.

Break: تنهي الحلقة أو switch فوراً وتنتقل إلى أول جملة بعد الحلقة.

Continue: تتخطى باقي الجمل في التكرار الحالي للحلقة وتنتقل إلى بداية التكرار التالي.

🔗 مثال على break و continue:

```
for (int i = 0; i < 128; i++){
    if (i == 120)
        break تساوي 120 ; // يخرج من الحلقة عندما
    if (i == 100 || i == 101)
        continue تساوي 100 أو 101 ; // يتخطى التكرار عندما
    if (Character.isLowerCase((char)i))
        System.out.println("value: " + i + " character: " + (char)i);
}
System.out.println("-----")
```

Break label and continue label:

تستخدم labels مع break و continue للخروج من أو الاستمرار في حلقات متداخلة (nested loops) بشكل محدد.

- Break labelName: يخرج من الحلقة التي تحمل الاسم labelName.
- Continue labelName: يتخطى التكرار الحالي للحلقة الداخلية وينتقل إلى بداية التكرار التالي للحلقة الخارجية التي تحمل الاسم labelName.

🔗 مثال على break label و continue label:

```
outer: // label الخارجية للحلقة
    for (; true ;) { // infinite loop
inner: // label الداخلية للحلقة
    for (int i = 0; i < 10; i++) {
        System.out.println("i = " + i);
        if (i == 2) { System.out.println("continue"); continue; } // يستمر في inner loop
        if (i == 3) { System.out.println("break"); i++; break; } // يخرج من inner loop
        if (i == 7) { System.out.println("continue outer"); i++; continue outer; } // يستمر في outer loop
    }
}
```

```

if (i == 8) { System.out.println("break outer"); break outer; } // من يخرج
for (int k = 0; k < 5; k++) {
    if (k == 3) { System.out.println("continue inner"); continue inner; } // في يستمر
    الأعق loop
}
}
}

```

معاملات الإسناد المركبة (Compound Assignment Operators):

توفر اختصاراً للعمليات التي يتم فيها تحديث قيمة متغير ثم إسناد القيمة الجديدة إليه.

- $c += 7$ يكافئ $c = c + 7$
- $d -= 4$ يكافئ $d = d - 4$
- $e *= 5$ يكافئ $e = e * 5$
- $f /= 3$ يكافئ $f = f / 3$
- $g \% = 9$ يكافئ $g = g \% 9$

معاملات الزيادة والنقصان (Increment and Decrement Operators):

++ (Increment): يزيد قيمة المتغير بمقدار 1.

- Prefix increment (++a): يزيد قيمة a ثم يستخدم القيمة الجديدة في التعبير.
- Postfix increment (a++): يستخدم القيمة الحالية ل a في التعبير ثم يزيد قيمتها.

-- (Decrement): ينقص قيمة المتغير بمقدار 1.

- Prefix decrement (--b): ينقص قيمة b ثم يستخدم القيمة الجديدة في التعبير.
- Postfix decrement (b--): يستخدم القيمة الحالية ل b في التعبير ثم ينقص قيمتها.

المعاملات المنطقية (Logical Operators):

يمكنك من تكوين شروط أكثر تعقيداً عن طريق دمج شروط بسيطة.

- && (Conditional AND): يكون الناتج True فقط إذا كان كلا الشرطين true.
- || (Conditional OR): يكون الناتج true إذا كان أحد الشرطين على الأقل true.
- & (Boolean logical AND bitwise): يستخدم أيضاً كمعامل مشابه ل && ولكن يقيم دائماً كلا الطرفين.
- | (Boolean logical inclusive OR): يستخدم أيضاً كمعامل مشابه ل || ولكن يقيم دائماً كلا الطرفين.

- $\wedge$ (Boolean logical exclusive OR): يكون الناتج true إذا كان أحد الشرطين true والآخر false يستخدم أيضاً كمعامل (bitwise).
 - ! (Logical NOT): يعكس قيمة الشرط البولياني (والعكس false تصبح true).
 - أسبقية المعاملات (Operator Precedence): يوجد ترتيب محدد لتقييم المعاملات المختلفة (الحسابية، العلائقية، المنطقية، الإسناد، إلخ....).
- 6- مخططات النشاط (UML Activity Diagrams): تستخدم لتمثيل تدفق التحكم في البرنامج بشكل مرئي، بما في ذلك التسلسل والاختيار والتكرار.

انتهت المحاضرة